

Linux Part 1

Introduction

- **Linux** is a **free and open-source operating system** kernel that serves as the core of many operating systems. It manages **hardware, memory, processes, and file systems**, allowing software applications to run.
- Linux is **stable, secure, and highly customizable**, which makes it popular for servers, desktops, and embedded systems.
- Unlike Windows or macOS, Linux is **open-source**, meaning anyone can view, modify, and distribute the code.

Linux Distributions

A **Linux distribution (distro)** is an operating system built around the Linux kernel, often including:

- Software packages (like office apps, browsers, and development tools)
- Package managers (for installing/removing software)
- A desktop environment (like GNOME, KDE, or XFCE)

Popular Linux Distributions

Distro	Description	Target Users
Ubuntu	User-friendly, easy installation	Beginners, desktops, servers
Debian	Stable and secure	Servers, developers
Fedora	Cutting-edge packages	Developers, advanced users
CentOS / AlmaLinux / Rocky Linux	Enterprise-focused, stable	Servers, enterprise environments
Arch Linux	Minimalist and highly customizable	Advanced users
Linux Mint	User-friendly, similar to Windows	Beginners switching from Windows

Who Created Linux?

Linux was created by **Linus Torvalds** in **1991**.

- He released the **Linux kernel** as open-source while he was a student at the University

of Helsinki, Finland.

- Since then, **thousands of developers worldwide** have contributed to the Linux kernel and its distributions.

Some Linux Commands

Command	Description
ls	List the current working directory
pwd	Print the current working directory
man	Show the manual page of the specific command some linux commands does not have manual page
cd	Change the current working directory
type -a	Show the files/directories of the command on the system type -a ls type -a pwd some commands does not have a directories, because it is a shell built-in like bg, cd,
whoami	Print the username associated with the current effective user ID
hostname	The command can get or set the hostname for NIS domain name
touch	Used to change the timestamp of the file, usually use this to make files, if the file exists will change the timestamp of the file. touch main.py touch index.html
echo	Print text, variables on the screen echo "hello world"
history	Show the previous commands that written on the shell
ip address	Show the ip address, show information about the IP address
mkdir	Make directories, or sub-directories # make empty directory mkdir test # make sub-directories mkdir -p /home/youssef/Desktop/test/test2/

rmdir	Remove the directories if they are empty rmdir /home/youssef/Desktop/test/
rm	Remove directories, by default does not remove non-empty directories, use <code>-recursive</code> to remove each listed directories # remove sub-directories rm -r /home/youssef/Desktop/test/test2
cat	Concatenate files, usually use this to show the file content on the terminal cat index.html cat main.py # show the line number cat -n main.py
cp	Used to copy files, directories from the source to destination cp test /home/youssef/Desktop
ps	Show all processes that are running
nano	Text editor, is a small and friendly text editor
which	Used to get the directory of the command on the machine get the path of the command on device which nano which ls
printf	Prints text in a formatted way, giving you more control than echo printf "Hello, World!\n"

Some Tips and Tricks

Brace Expansion

- touch file{4..10} # create files from file4 to file10
- mkdir dir{1..10} # create directories from dir1 to dir100
- ls file{10..100} # will show files from file1 to file10
- ls file[1-3] # will display files from file1 to file3
- ls file[!2] # will display all files, files ends with two digits like file10
- ls file[!1]* # will exclude number one on the file name

- `ls [!a-c]*` # match files NOT starting with certain letters
- `ls *[!t]` # match files NOT ending with `t`
- `ls file[0-9][0-9]` # will list files with two digits
- `ls *.txt` # will displays all files with extension → `.txt`
- `ls file?` # means will displays all files with one characters after “file” keyword

`[]` → used for character matching

`{}` → used for brace expansion

`*` → refers to all matches

`!` → called Globbing, NOT

`?` → exactly one character

Shell Variables

first you must define the variable then assign the value, the variable mustn't include any space

- `age=25`
- `name="ahmed"`

print the variables of get the variables

- `echo $name`
- `echo $age`

Take the value of the variable form command

- `currentDate=$(date)`

Some Built-in Variables

Variable name	description
<code>\$PATH</code>	Is an environment variable that tells the shell where to look for executable programs when you type command. When you type a command like <code>ls</code> or <code>python</code> , Linux searches for that program in the directories listed in <code>\$PATH</code> .
<code>\$USER</code>	Is a variable that stores the username of the currently logged-in user
<code>\$HOME</code>	Path to the user's home directory
<code>\$HOSTNAME</code>	Computer's network name
<code>\$SHELL</code>	Default login shell
<code>\$PWD</code>	Present working directory
<code>\$OLDPWD</code>	Previous working directory

\$LANG	Language/locale setting
\$PPID	Parent process ID
printenv	Show all environment variables

What is vim?

- Short for vi improved, is a text editor used on the terminal
- it's exactly fast, powerful, and perfect for editing code or configuration files directly from the command line
- vim is available by default on almost all linux systems
- create file using vim → **vim filename**
→ if the file exists will open it, if the file does not exist will create it

vim Modes

- vim has three modes
- 1. normal modes → default mode when you open vim
- 2. insert mode → press **i** to enter, **Esc** to exist
- 3. command mode → saving, quitting, searching, to enter to it press **:** from normal mode

Command Mode

Command	meaning
:w	Save [write] file
:q	Quit if no changes made
:wq	Save and quit
:x	Save and quit, like to → :wq
:q!	Quit without saving
:set number	Show line number
:set number	Hide line number
:/word	Search for word
:help command	Open vim help about that command

Normal Mode

- to go to normal mode press

Command	Meaning
h	Move left
l	Move right
k	Move up
j	Move down
0	Start of line
\$	End of line
gg	Move to top of file
G	Move to bottom of the file
w	Move forward one word
b	Move backward one word
x	Delete character
dw	Delete word
yy	Copy line
p	Paste below, paste above
u	Undo
Ctrl + r	Redo
n	Next match
N	Previous match
i	Enter Insert mode
I	Enter insert mode at line start
A	Enter insert mode at line end

Whats is Nano?

- Nano is a simple text editor that runs inside the Linux terminal. It lets you create, edit, and save text files (like code, configs, or notes) directly from the

command line.

- It's easier to use than editors like vi or vim, which are more advanced.
- Create file using nano: nano filename.txt

Basic nano Commands

Action	Key	Description
save	Ctrl + o	Save changes to file
Exit	Ctrl + x	Close nano
Cut text	Ctrl + k	Cut the current line
Paste text	Ctrl + u	Paste previously cut text
Search	Ctrl + w	Find text in the file
Go to line	Ctrl + _	Jump to specific line
Undo / Redo	Alt + U / Alt + E	Undo or redo last change
Help Menu	Ctrl + G	Show Nano's built-in help

The outlines key principles guiding Unix system design and more badly, good software engineering practice.

The Nine Major Tenets

- 1) small is beautiful
- 2) each program does one thing well
- 3) prototype as soon as possible
- 4) choose probability over efficiency
- 5) store data in flat text files
- 6) use software leverage
- 7) use shell scripts to increase leverage and probability
- 8) avoid captive user interface
- 9) make every program a filter

what the difference between shell and terminal?

1.Terminal

- A terminal is a program(window) that lets you interact with the system through text
- it is basically the interface - the screen where you type commands and see the results
- think of the terminal as the doorway to talk to your computer
- examples of terminal programs
 - GNOME Terminal (on ubuntu), konsole, xterm, windows terminal,

2.Shell

- the shell is the program running inside the terminal that actually interprets your commands
- it understands what you type (like ls, pwd, mkdir, ...), and tells the operating system

- what to do
- think of the shell as the brain that understands and executes your commands
- common linux shells
 - bash (most common), zsh, fish, sh, ksh

show all available shells on the system

- `ls /bin/*.sh`

show the shell that you username

- `echo $SHELL`
- `cat /etc/shell`

change the shell

- `chsh -s /bin/zsh` # change shell permanently
- `dash` # change shell temporarily
- `sh` # change shell temporarily

when you write ls

- the terminal displays the text
- the shell runs the command and print the output

Ctrl + alt + F6

- Can you change your terminal to shell only without any GUI
 - To escape this terminal press **[ctrl + alt + F2]**
- that switches you to a different **TTY (terminal)** – a full screen text console outside your graphical desktop
- to exit from the text-mode write `ctrl + alt + F2`
- the last GUI with all apps, .. is a one terminal on the kernel
- also can change your terminal by using **ctrl + alt + F2**

Connect To Remote Linux Server

- `ssh sami@localhost`
 - ssh stands for secure shell – a protocol that lets you connect securely to another computer (usually server)
 - sami refers to the username you're logging in with
 - localhost refers to the the host you are connecting to – here it means your own computer (127.0.0.1)
- **syntax**
 - `ssh username@server_ip_or_domain`
 - `ssh youssef@myserver.com`

Option	Example	What it Does
-p	<code>ssh -p 2222 user@server</code>	Connect on a custom port (default is 22)
-i	<code>ssh -i ~/.ssh/id_rsa user@server</code>	Use a specific private key file

Option	Example	What it Does
-v	ssh -v user@server	Verbose mode (for debugging connection)

Command Chaining

- means run multiple command on the same line on the terminal
- echo \$PATH; ls; pwd;

Make Variable Global Variable

- save the variables on buffer (memory)
 - export \$var
 - export \$name

Show all variables that exported, that can be used in any terminal

- declare -x

Create Variables using declare

- declare -i num1 # create variable as integer
- declare -i num2 # create variable as integer

expr

- is a command line utility in linux used to evaluate expressions – usually arithmetic operations, string operations, or comparison
- examples
 - expr 5 + 3
 - expr 10 - 4
 - expr 6 * 7
 - expr 5 = 5
 - expr 5 \< 10 # The < must be escaped (\<) in bash.
 - expr length "Hello" # string operation
 - expr substr "HelloWorld" 1 5 # extract sub-string

how to make the variables permanent even when restart the machine?

- Declare the variables inside the shell startup files
- shell startup files: are scripts that the shell automatically runs when you start a new shell session. They help configure the shell environment, set variables, define aliases, and customize your experience. They setup settings like:
 - environment variables (\$PATH, \$EDITOR, etc)
 - alias (ll='ls -la')
 - functions
 - shell options
- there are two types of shells

1.login-shell:

- -bash → login shell (contains dash before bash keyword)
- example: ssh login, localhost login

2.non-login shell:

- bash → non-login shell
- example: normal terminal emulator

what the meaning of \$0

- is a special shell variable in linux that represent the name of the currently running shell or script
- it is read only, you can't assign a value to it

let declare variables on the startup files

- `sudo nano /etc/profile` # this is the first file to add the variables at startup
 - any change to this file will appear when open login-shell
 - `sudo nano ~/.bashrc`
 - this for login-shell and non-login shell
-

Linux Part 2

Pipes Redirection Filters

ls -l ~

- the previous command will list the directories that exists on the home directory
- `-l` → called token
- `~` → called token
- each token is separated by the space
- all these token after pressing enter called **Standard Input**
- the output of the command called **Standard Output**
- to check if the latest command is run successfully or not, write the following command
→ `echo $? [exit code must equal 0]`

Example

`ls -l ~` → this called standard input

the following is standard output

```
total 60
drwxr-xr-x 7 youssef youssef 4096 Nov 18 13:32 Desktop
drwxr-xr-x 2 youssef youssef 4096 Nov 17 01:48 Documents
drwxr-xr-x 11 youssef youssef 4096 Nov 18 11:46 Downloads
drwxr-xr-x 2 youssef youssef 4096 Nov 17 01:48 Music
drwxr-xr-x 4 youssef youssef 4096 Nov 18 13:34 Pictures
drwxr-xr-x 2 youssef youssef 4096 Nov 17 01:48 Public
drwxr-xr-x 2 youssef youssef 4096 Nov 17 01:48 Templates
drwxr-xr-x 2 youssef youssef 4096 Nov 17 01:48 Videos
```

How To Redirect Output Of Command Inside File?

- Run the following command
 - `man ls > ls` → called standard output
 - then show the file content, we get the manual page inside the file

note that: `>` this used to override if the file name exists, and `>>` this used to append to the file

Redirect The Error On File

- `ls /err 2> error` → this called standard error
- `ls /home/ 1> ls` → 1 refers to the command runs successfully

Can Mix Between These

- `cat /etc/passwd /error 1> passwords 2> errors` → this mix between standard output and error

Standard Input

- usually show the file content on the terminal
- `cat < error`
- usually you will never use this command

what is pipes?

- Each output of any command/shell can be input to another input
- example using **grep** command
 - **grep ff00 hosts**
 - the result will be as the following → `ff00::0 ip6-mcastprefix`
 - **grep -l ff00 hosts** → used to show the path of the file → hosts
 - **grep -i -n ff00 hosts** → used to show the line number of the result
 - **grep -r Port /etc/** → will search inside all directories and sub-directories about the string and get the result as red words, and will show the path of the file
 - **grep -i port hosts** → will ignore the case sensitive [port = PORT = Port ...]
 - **grep -v port hosts** → will get all lines that does not contains any word = port, v → refers to invert the result
 - **grep -rc hosts** → will search inside the directories, and sub-directories, then get the number of repeated lines

Show The First 10 Lines Of The Text File

- **head file**

You Can Determine The Number Of Lines

- **head -n 5 file**

Show the last 10 lines of the file

- tail file

you can determine the number of lines of on the file

- **tail -n 5 file**

You Can Count The Words On The File

- **wc /etc/hosts** → will count number of lines, count words, and number of characters

→ Search About Word At The First 50 Lines On Specific File. In This Situation We will Use Pipe.

- **head -n 50 /var/log/syslog | grep -i "intel"**

- **Get The Number Of Lines**

→ **head -n 50 /var/log/syslog | wc -l**

- **Get The Number Of Words**

→ **head -n 50 /var/log/syslog | wc -w**

- **Get The Number Of Characters**

→ **head -n 50 /var/log/syslog | wc -c**

Count Number Of Repeated Word Inside Specific Output of File

- **head -n 50 /var/log/syslog | grep "kernel" | wc -w**

What is tr command?

- is a simple but powerful tool used to translate, substitute, or delete characters from input. It reads from standard input (stdin) and writes the result to standard output (stdout).
- Example to delete specific character from the given word, sentence
 - **echo "this is a test 123" | tr -d 's'** → output: thi i a tet 123
- Example to convert character to another characters
 - **echo "this is a test 123" | tr [i] [I]** → output: thIs Is a test 123
- Remove repeated character based on specific number
 - **echo "this iss a test 123" | tr -s 's'** → output: this is a test 123
- You can use regular expressions
 - **echo "this iss a test 123" | tr [a-z] [A-Z]** → output: THIS ISS A TEST 123
- Delete all digits from the text
 - **echo "this iss a test 123" | tr -d [:digit:]** → output: this iss a test
- Replace any digit to character
 - **echo "this iss a test 123" | tr [:digit:] 'z'** → output: this iss a test zzz
- You can use files with tr command by using cat command
 - **cat /etc/hosts | head -n 2 | tr [a-z] [A-Z]**
 - Output of the command
 - 127.0.0.1 LOCALHOST
 - 127.0.1.1 YOUSSEF

What cut command ?

- is used to extract specific sections of text from each line of input—usually fields

(columns) or character positions.

- It is extremely useful when working with logs, CSV files, or any text with a delimiter.

- Examples of cut command

- **echo "this is a string of bytes" | cut -c 1** → output: t
 - This get the first character of the output
- **echo "this is a string of bytes" | cut -b 1** → output: t
 - This get the first byte of the output
- **echo "this is a string of bytes" | cut -c 2,21** → output: hb
 - Get specific characters from string by using its index
- **echo "this is a string of bytes" | cut -c 2-10** → output: his is a
 - Get range of characters from the text
- **echo "this is a string of bytes" | cut -f 1 -d " "** → output: this
 - Get the first column from the text, must determine the delimiter [-d]
 - Delimiter splits the string into fields
- **cat /etc/hosts | cut -d " " -f 1** → get the first column from the file, output as following:
 - 127.0.0.1
 - 127.0.1.1
 -
 - #
 - ::1
 - fe00::0
 - ff00::0
 - ff02::1
 - ff02::2
- **cat /etc/passwd | head -n 3 | cut -f 1 -d ":"** → get the first 3 users that exists on the Linux system
 - Output as the following
 - root
 - daemon
 - bin

ChatGPT Summary

The Standard Streams

Stream Name	Abbreviation	File Descriptor	Default Device	Purpose	Example Use
Standard Input	STDIN	0	Keyboard	Provides input <i>to</i> commands/programs	cat < file.txt
Standard Output	STDOUT	1	Terminal (screen)	Shows normal output <i>from</i> commands	ls > out.txt

Stream Name	Abbreviation	File Descriptor	Default Device	Purpose	Example Use
Standard Error	STDERR	2	Terminal (separate from STDOUT)	Shows error messages	ls fakefile 2> error.txt

Linux Streams Examples

Stream Name	Abbreviation	File Descriptor	Default Device	Purpose	Example Command	Example Explanation
Standard Input	STDIN	0	Keyboard	Input to a program	sort < names.txt	Takes input from names.txt instead of keyboard
Standard Output	STDOUT	1	Terminal	Normal program output	ls > out.txt	Sends output to out.txt instead of screen
Standard Error	STDERR	2	Terminal	Error messages	ls fakefile 2> errors.txt	Sends errors to errors.txt , not the terminal
STDOUT + STDERR together	—	1 & 2	Terminal	Combine normal + error output	command &> all.txt	Saves both normal output and errors to one file
Append STDOUT	—	1	Terminal	Add output to end of file	echo "hi" >> log.txt	Adds “hi” to log.txt without overwriting
Append STDERR	—	2	Terminal	Add errors to end of file	command 2>> err.log	Appends only errors to err.log
Discard STDOUT	—	1	/dev/null	Hide all normal output	command > /dev/null	Output is deleted (not shown, not saved)
Discard STDERR	—	2	/dev/null	Hide errors	command 2> /dev/null	Error messages are discarded
Discard both	—	1 & 2	/dev/null	Hide everything	command > /dev/null 2>&1	No output, no errors
Pipe (uses STDOUT → STDIN)	—	1 → 0	Terminal/ next command	Send output of one command	`ls`	grep ".txt"

Stream Name	Abbreviation	File Descriptor	Default Device	Purpose	Example Command	Example Explanation
				into another		

What the meaning of the redirection?

- Redirection is the process of sending input/output of a command to something other than its default destination.

✓ Redirect STDOUT → a file

- `ls > files.txt`

✓ Redirect STDERR → a file

- `ls fakefile 2> error.log`

✓ Redirect STDIN ← from a file

- `sort < names.txt`

✓ Redirect both STDOUT + STDERR

- `command > all.log 2>&1`

✓ Discard output

- `command > /dev/null`

Why is Powerful?

- Save output to a file
- log errors
- read input automatically from files
- combine commands
- clean or silence output
- script and automate tasks

linux p3

Linux Part 3

User and Group Management

